

Memory Matching Flutter Game

CS5450 Mobile Programming - Exercise 2

Instructor	Dr. Sabah Mohammed
Student	Luqman Aadil
Repository	https://github.com/luqi101/Mobile_Programming_Exercise_2.git
Date Generated	May 20, 2026

Formal Submission Document

This document describes the final Flutter/Dart application, its functionality, configuration, verification, screenshots, and project structure for CS5450 Exercise 2.

Executive Overview

Memory Match is an original Flutter/Dart memory matching game created for CS5450 Exercise 2. The player selects a difficulty level, flips cards to locate matching pairs, monitors elapsed time and move count, and receives a calculated score after completing the board. The implementation combines local image assets, a responsive card grid, structured Dart models, and controller-driven game logic to provide a complete mobile programming submission.

The application is designed to run as a Flutter project with Android, iOS, and Web platform folders. The primary target for demonstration is Android emulator or physical Android device, with Chrome/Web also supported for additional verification.

Assignment Requirements Coverage

Assignment Requirement	Project Implementation
Flutter/Dart implementation	Application source is implemented under lib/ using Flutter widgets, Dart models, reusable widgets, and a controller-driven game flow.
Memory matching functionality	Cards are generated in pairs, shuffled, flipped, matched, and reset after mismatches. Win state is detected after all pairs are matched.
Configuration and run instructions	This document and README.md include setup, Android, Chrome/Web, analysis, and test commands.
Screenshots	Real screenshots from the screenshots/ folder are included with captions.
Exact project structure	A project tree is included, excluding build outputs, caches, Git metadata, and unnecessary generated internals.
Local images/assets	Local PNG card and background assets are stored under assets/images and registered in pubspec.yaml.
Testing and verification	Automated controller and widget tests are included under test/. The latest recorded verification shows passing analysis and test results.

Application Features

- A polished home screen provides difficulty selection before the game starts.
- The game screen renders cards in a responsive grid that adapts to different screen sizes.
- Card selection supports flip, match, mismatch, and automatic reset behavior.
- Input is locked during mismatch delay to prevent rapid-tap state errors.
- The header displays timer, moves, matched-pair progress, restart, and home actions.
- The result screen displays difficulty, final time, moves, pairs, score, and replay navigation.
- Local generated card and background images are used instead of network assets.

Game Rules and Difficulty Levels

The player flips two cards at a time. If both cards represent the same image, the pair remains visible and contributes to the matched-pair count. If the images do not match, both cards flip back after a short delay. Each two-card attempt counts as one move. The game ends when all pairs on the board have been matched.

Difficulty	Pairs	Cards	Purpose
Easy	4	8	Quick introductory board for demonstrating the core mechanic.
Medium	6	12	Balanced board for normal play and screenshot demonstration.
Hard	8	16	Larger board that demonstrates scalability and responsive layout.

Technical Implementation

The application is organized into app, controller, model, screen, widget, and utility layers. This separation keeps the user interface readable while keeping the game rules and state transitions inside the controller and model classes.

GameController is implemented as the central state object. It owns deck generation, card selection, move counting, matched-pair tracking, timer updates, restart behavior, mismatch locking, delayed card reset, and win detection.

Difficulty, MemoryCard, and GameResult provide a small model layer for board configuration, individual card state, and completed game results. Utility files isolate score calculation and duration formatting.

Reusable widgets under lib/widgets render the difficulty selector, game header, card tile, primary action button, background, and responsive grid. This keeps screen files focused on layout composition and navigation.

Assets are local PNG files under assets/images/cards and assets/images/backgrounds. The project does not depend on network images, which improves reliability during grading and offline demonstration.

Automated tests cover core controller behavior, matching and mismatch rules, restart behavior, score calculation, win detection, and the basic widget flow.

Setup and Run Instructions

Open the project at the repository root and install dependencies before running the application. The following commands are intended to be run from the root directory containing pubspec.yaml.

```
flutter pub get
dart format .
flutter analyze
flutter test
```

Android emulator or physical Android device:

```
flutter devices
flutter run -d <android-device-id>
```

Replace with the device identifier shown by flutter devices. This is the recommended demonstration target for the mobile programming exercise.

Chrome/Web run command:

```
flutter run -d chrome
```

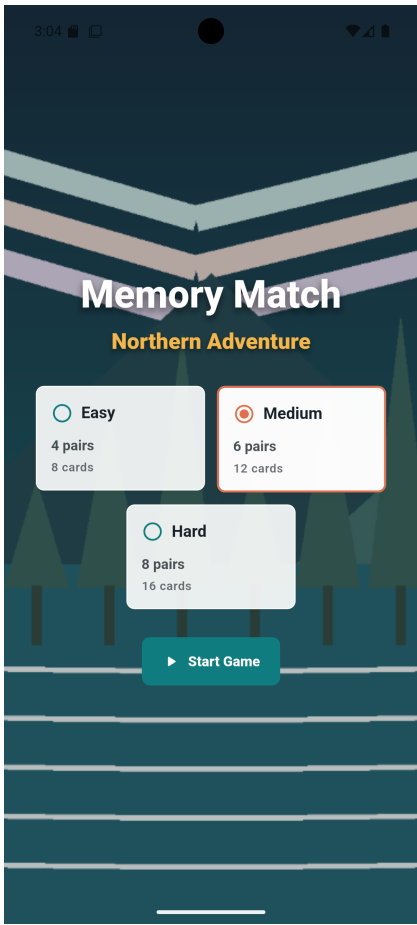
Testing and Verification

The latest verification information recorded for the project is summarized below. These commands should be rerun after any future change to the source code, assets, tests, or project configuration.

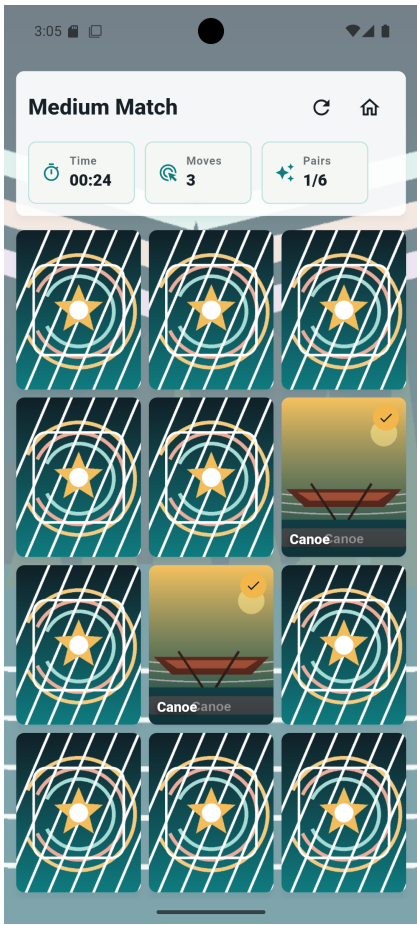
Command	Purpose	Latest Recorded Result
flutter clean	Remove previous build outputs before final verification.	Pass
flutter pub get	Resolve Flutter/Dart dependencies.	Pass
dart format .	Apply Dart formatting across the project.	Pass, no changes required
flutter analyze	Run static analysis and detect code/configuration issues.	Pass, no issues found
flutter test	Run automated controller and widget tests.	Pass, 13 tests
flutter devices	List available Android/Web targets.	Android emulator and Chrome detected

Screenshots

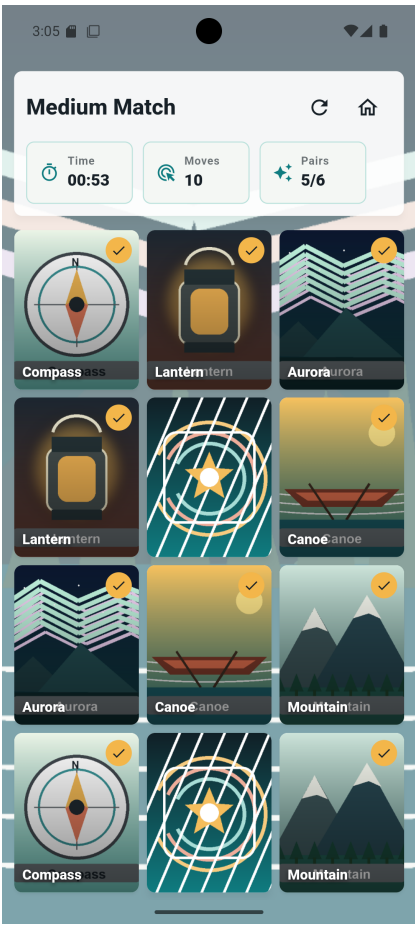
The following screenshots are included from the screenshots/ folder and show the main application flow: home screen, active game state, game progress, and completion result.



Home screen and difficulty selection



Active game screen with matched pair progress



Game progress showing multiple matched pairs



Win/result screen with final time, moves, pairs, and score

Exact Project Structure

The structure below reflects the project organization used for the submitted Flutter application. Build outputs, cache folders, Git metadata, IDE caches, and temporary operating-system files are intentionally excluded. Generated platform folders are summarized at a readable level, while the application source, assets, screenshots, tests, and documentation files are listed explicitly.

```
.
|-- android/                Flutter Android platform project
|-- ios/                   Flutter iOS platform project
|-- web/
|   |-- icons/
|   |-- favicon.png
|   |-- index.html
|   |-- manifest.json
|-- assets/
|   |-- images/
|       |-- backgrounds/
|           |-- game_background.png
|           |-- home_background.png
|       |-- cards/
|           |-- card_aurora.png
|           |-- card_back.png
|           |-- card_canoes.png
|           |-- card_compass.png
|           |-- card_lake.png
|           |-- card_lantern.png
|           |-- card_mountain.png
|           |-- card_pine.png
|           |-- card_star.png
|-- lib/
|   |-- app/               app shell and theme
|   |-- controllers/       GameController state and rules
|   |-- models/            Difficulty, MemoryCard, GameResult
|   |-- screens/           Home, game, and result screens
|   |-- utils/             formatting and scoring helpers
|   |-- widgets/           reusable UI components
|   |-- main.dart
|-- screenshots/
|   |-- Screenshot_20260520_030511.png
|   |-- Screenshot_20260520_030550.png
|   |-- Screenshot_20260520_030619.png
|   |-- Screenshot_20260520_030629.png
|-- test/
|   |-- game_controller_test.dart
|   |-- widget_test.dart
|-- tools/
|   |-- build_readme_pdf.py
|   |-- generate_assets.py
|-- .gitignore
|-- .metadata
|-- AGENTS.md
|-- analysis_options.yaml
|-- pubspec.lock
|-- pubspec.yaml
|-- README.md
|-- README.pdf
```